

High-Level Design Document for Caching for LLM Project

Author: Adi Volfman, Tomer Eliel

Version: V0.1

Created: 17/05/26

Last Updated: 21/05/26

1. Introduction

This document describes the high level design of the **Caching for LLM** project. The project implements a semantic cache for Large Language Model queries. Instead of caching only exact prompt strings, the system converts a user prompt into an embedding vector, searches a vector store for semantically similar previous prompts, and returns a cached response when the similarity is high enough. If no suitable cached response exists, the system forwards the prompt to an LLM service, stores the generated response in the vector cache, and returns it to the client.

The system is designed as a set of independent services that communicate using gRPC. Each major component can run as a separate local process, as a container, or on a different machine. Inside each service, concrete providers are hidden behind internal interfaces. For example, the Embedding Service owns an object implementing IEmbedder, and the initial implementation may be OllamaEmbedder. This allows us to replace Ollama with another embedding provider later without changing the orchestrator or the external gRPC contract.

The first version focuses on the core semantic caching flow:

1. Receive a prompt from the client.
 2. Generate an embedding for the prompt.
 3. Search the vector cache for a similar embedding.
 4. Return a cached response on a cache hit.
 5. Query the LLM on a cache miss.
 6. Store the new embedding response pair in the vector store.
 7. Return the final response to the client.
-

1.1 Overview

The system is composed of the following main processes:

- **Client CLI Process** - receives user input and displays the response.
- **Cache Orchestrator Process** - coordinates the full request lifecycle.
- **Embedding Service Process** - converts text prompts into embedding vectors.
- **Vector Store Service Process** - stores embedding response pairs and performs similarity search.
- **LLM Service Process** - generates answers using an LLM provider.

The orchestrator is the central service. The client communicates only with the orchestrator. The orchestrator communicates with the other components through gRPC. Provider specific logic is located inside the relevant service process. Therefore the orchestrator does not directly depend on any concrete provider.

A key design principle is separation between two kinds of interfaces:

1. **External service interfaces** - gRPC APIs used for communication between processes and the different components.
2. **Internal provider interfaces** - interfaces/classes used inside each service to support pluggable providers.

For example, the Embedding Service exposes a gRPC method such as `Embed(EmbedRequest) -> EmbedResponse`, while internally it delegates the work to an `IEmbedder` instance. The first concrete implementation can be `OllamaEmbedder`, but future implementations may include another embedding provider or a mock embedder for testing.

1.2 Assumptions and Design Constraints

Assumptions

- Prompts are valid text inputs.
- The embedding service is reachable and returns a valid numeric vector for each prompt.
- The vector store supports insertion and nearest neighbor similarity lookup.
- The LLM service is reachable and can generate a textual response for a prompt.
- The cache has a configurable maximum capacity.
- The similarity threshold is configurable.
- In the first demo version, one concrete provider per service is sufficient, as long as the architecture allows replacement.
- Each major component can run as a separate process.

Design Constraints

- Components must communicate through well defined APIs and not through direct shared memory.
 - The system should support local execution and later remote/cloud deployment.
 - The orchestrator must not contain provider specific implementation details.
 - Provider replacement should require configuration/code changes inside the relevant service only, not changes to the entire system.
 - gRPC is selected as the main inter process communication mechanism because it provides strongly typed contracts, generated client/server stubs, efficient binary serialization, and a clean RPC model.
-

1.3 Distributed and Parallel Aspects

The project has an inherent distributed nature because the system is intentionally decomposed into independent services: client, orchestrator, embedding service, vector store service, and LLM service.

This decomposition is useful because each component represents a different responsibility and may have different resource requirements. For example, the LLM and embedding services may require access to model runtimes such as Ollama, while the vector store service may require persistent storage and efficient similarity search.

Compared to a monolithic implementation, the distributed design introduces several difficulties. The system must handle network failures, unavailable services, service discovery, timeouts and more. In a monolithic program, these components could call each other directly through function calls, but in this design every service boundary requires a gRPC request and response.

We plan to deal with these difficulties by using strongly defined gRPC service contracts, centralized configuration, timeouts, and clear ownership of responsibilities. The orchestrator will be responsible for request coordination, while each service will be responsible only for its own domain. The vector store service will be the single owner of cache state, which helps prevent inconsistent cache updates.

This project is interesting as a distributed system because it demonstrates a realistic pattern used in modern AI applications: an orchestration layer coordinates several independent model/storage services. The design allows components to be scaled or replaced independently. For example, the embedding service can later be moved to a GPU machine.

1.4 Dependencies

Dependency	Planned Use	Initial Choice	Backup / Alternative
gRPC	Communication between processes	grpcio, grpcio-tools	REST/HTTP API if gRPC becomes too complex
Protocol Buffers	Define service contracts and message schemas	.proto files	JSON schemas for REST fallback
Embedding provider	Generate embedding vectors from prompts	Ollama with an embedding model such as embeddinggemma	BGE - M3, mock embedder
LLM provider	Generate responses on cache misses	Ollama with a Gemma family model	OpenAI API, another local model, mock LLM
Vector store	Store embeddings and perform similarity search	Chroma	In memory vector store for testing, FAISS
Docker	Run services in isolated containers	Docker Compose for local deployment	Manual process startup
Python	Main implementation language	Python 3.x	N/A

1.5 Issues List

Open issues to decide during design and implementation:

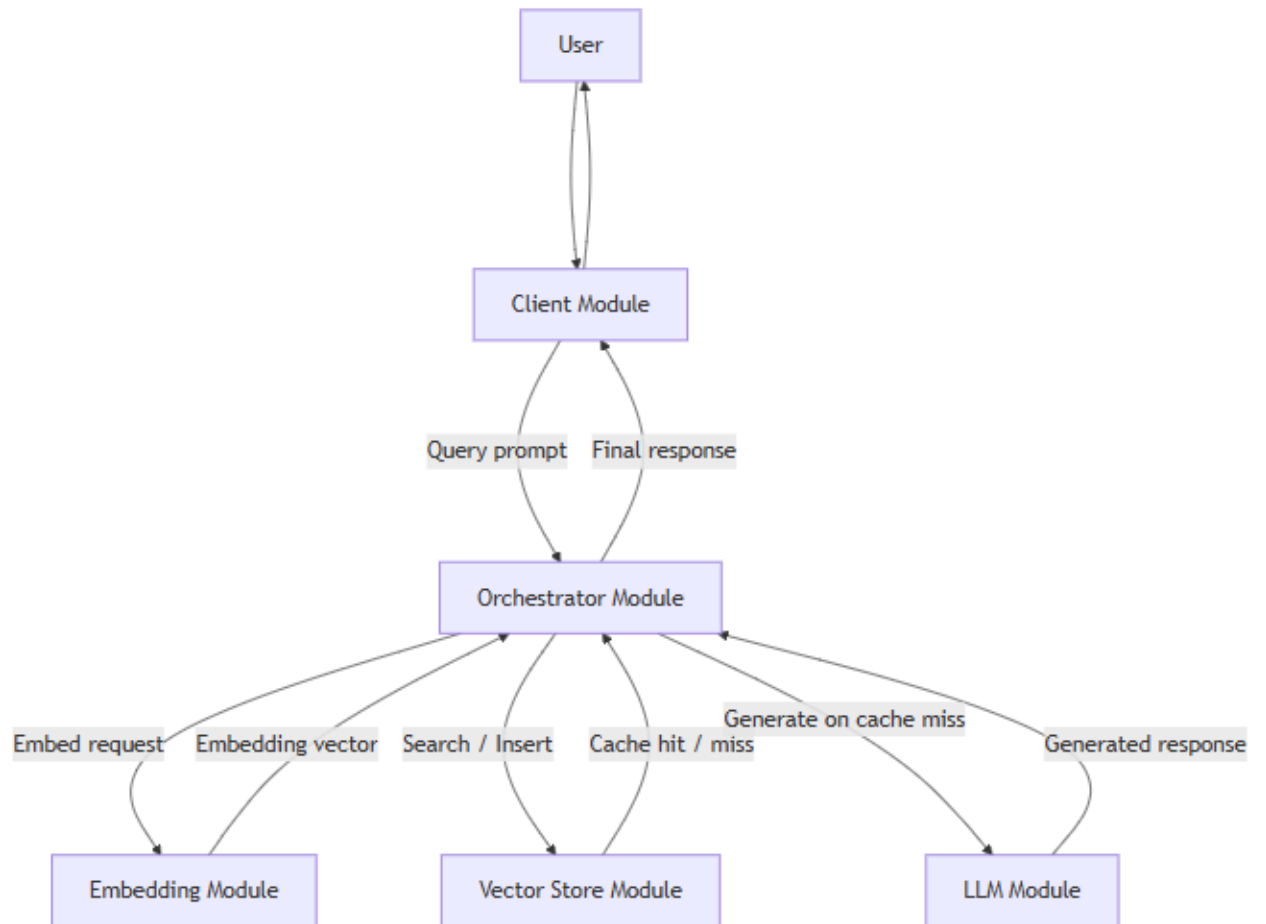
1. Exact similarity metric: cosine similarity, Euclidean distance, or vector store default.
2. Initial threshold value for cache hit decisions.
3. Tie breaking policy when multiple cached entries have the same similarity score.
4. Initial eviction policy: always admit + LRU, LFU, or simple FIFO.
5. Whether the orchestrator should insert into the cache synchronously before returning the response, or asynchronously after returning the response.
6. How much metadata should be returned to the client: cache hit/miss, score, provider name.
7. How remote services will be discovered: static configuration file, environment variables, or command-line arguments.

1.6 To-do List and Expected Timetable

Task	Description	Estimated Time
Define gRPC contracts	Write .proto files for orchestrator, embedding, vector store, and LLM services	1–2 days
Implement service interfaces	Define IEmbedder, IVectorStore, ILLMProvider, IEvictionPolicy	1 day
Implement Embedding Service	gRPC server + OllamaEmbedder implementation	2–3 days
Implement LLM Service	gRPC server + OllamaLLMProvider implementation	2–3 days
Implement Vector Store Service	gRPC server + Chroma or in-memory implementation	3–4 days
Implement Orchestrator	Full cache hit/miss flow and service clients	3–4 days
Implement CLI Client	Basic prompt input and response display	1–2 days
Add configuration	YAML/env/CLI based provider and endpoint configuration	1–2 days
Add Docker Compose	Local multi process setup	2–3 days
Add logging and error handling	Timeouts, retries, clear errors, health checks	2–3 days
Integration testing	Test full flow locally	2–3 days
Documentation and demo	Update design docs, README, demo commands	2–3 days

2. Logical Architecture

2.1 Architecture Diagram



2.2 Main Modules

2.2.1 Client CLI Module

The Client CLI is the user facing process. It receives prompts from the user, sends them to the orchestrator using a gRPC client stub, and displays the returned response.

Responsibilities:

- Read prompt text from the user.
- Send QueryRequest to the orchestrator.
- Display response text.
- Optionally display metadata such as cache_hit, similarity_score.

2.2.2 Cache Orchestrator Module

The Cache Orchestrator is the central coordination module. It receives prompts from the client and executes the semantic caching algorithm.

Responsibilities:

- Receive prompts from the client.
- Request embeddings from the Embedding Service.
- Query the Vector Store Service using the embedding and threshold.
- Decide whether the result is a cache hit or cache miss.
- On cache hit, return the cached response.
- On cache miss, request a new response from the LLM Service.
- Insert the new embedding-response pair into the Vector Store Service.
- Return the final response and metadata to the client.

The orchestrator should depend only on generated gRPC clients/stubs, not on provider-specific classes such as OllamaEmbedder or ChromaVectorStore.

2.2.3 Embedding Service Module

The Embedding Service receives text and returns embedding vectors. It exposes a gRPC API externally and uses an internal IEmbedder interface to support multiple embedding providers.

Responsibilities:

- Expose Embed gRPC method.
- Validate the input prompt.
- Delegate embedding generation to an IEmbedder implementation.
- Return the embedding vector and optional metadata.

Provider:

- OllamaEmbedder, using Ollama embedding API.

2.2.4 Vector Store Service Module

The Vector Store Service owns the cache state. It stores embedding response pairs and performs similarity search. It exposes gRPC methods for search and insert operations.

Responsibilities:

- Store cache entries.
- Search for the closest embedding to a query embedding.
- Apply the configured similarity threshold.
- Return cached response on hit.
- Insert new cache entries.
- Enforce cache capacity.
- Apply admission and eviction policies.

Provider:

- ChromaVectorStore, using Chroma as the backing vector database.

2.2.5 LLM Service Module

The LLM Service receives prompts and returns generated responses. It hides the concrete LLM provider behind an internal ILLMProvider interface.

Responsibilities:

- Expose Generate gRPC method.
- Validate prompt input.
- Delegate generation to an ILLMProvider implementation.
- Return generated text and optional metadata.

Provider:

- OllamaLLMProvider, using Ollama with a Gemma family model.

2.2.6 Configuration Module

The Configuration Module loads runtime settings from a configuration file, command-line arguments, or environment variables.

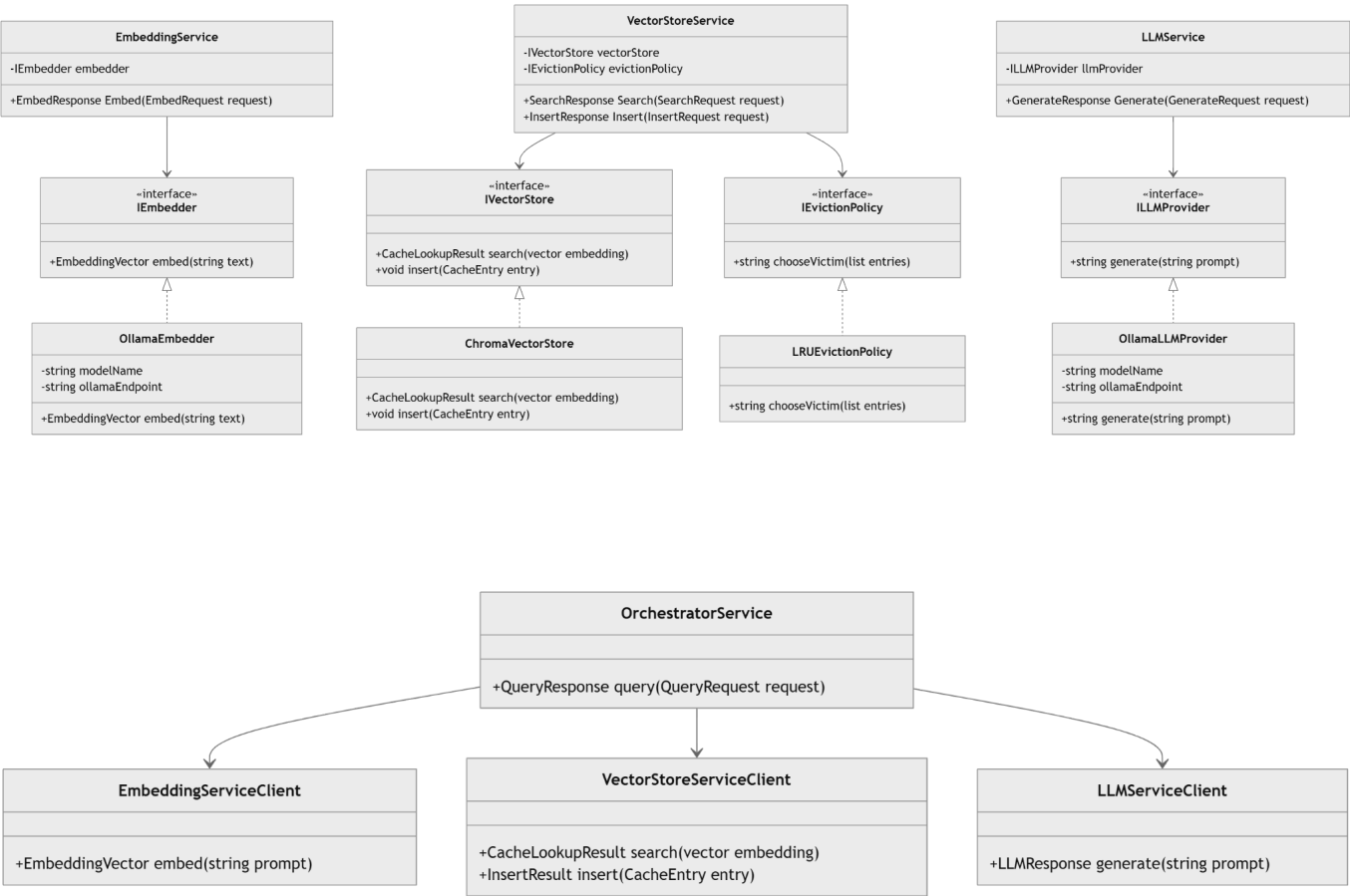
Responsibilities:

- Configure service endpoints.
 - Configure provider choices.
 - Configure model names.
 - Configure cache threshold and capacity.
 - Configure timeouts and retries.
-

3. Design

3.1 Modules / Main Classes

3.1.1 Class Diagram



3.1.2 Class Details

OrchestratorService

The main service class inside the orchestrator process. It implements the external Query gRPC endpoint used by the CLI client.

Main method:

query(prompt) -> response

High-level behavior:

1. Validate prompt.
2. Call EmbeddingServiceClient.embed(prompt).
3. Call VectorStoreServiceClient.search(embedding).
4. If hit, return cached response.
5. If miss, call LLMServiceClient.generate(prompt).
6. Insert new cache entry through VectorStoreServiceClient.insert(...).
7. Return generated response.

EmbeddingServiceClient

A client side wrapper used by the orchestrator to communicate with the Embedding Service process through gRPC.

It hides the generated gRPC stub details from the orchestrator business logic and exposes a simple method for requesting embeddings.

EmbeddingService

The gRPC server implementation inside the Embedding Service process. It owns an IEmbedder instance.

EmbeddingService(IEmbedder embedder)

It does not know whether the concrete provider is Ollama, OpenAI, or another model. It only calls the interface.

IEmbedder

Internal interface for embedding providers.

```
class IEmbedder(Protocol):  
    def embed(self, text: str) -> list[float]:  
    ...
```

OllamaEmbedder

Concrete implementation of IEmbedder. It calls the Ollama embedding API using a configured model name.

Responsibilities:

- Hold Ollama endpoint and model name.
- Send text to Ollama.
- Parse and return the embedding vector.
- Convert provider-specific errors into service-level errors.

VectorStoreServiceClient

A client side wrapper used by the orchestrator to communicate with the Vector Store Service process through gRPC.

It hides the generated gRPC stub details from the orchestrator business logic and exposes a simple interface for cache lookup and cache insertion.

VectorStoreService

The gRPC server implementation inside the Vector Store Service process. It owns an IVectorStore implementation and optionally an IEvictionPolicy.

Main methods:

Search(embedding) -> CacheLookupResult

Insert(embedding, prompt, response, metadata) -> InsertResult

IVectorStore

Internal interface for vector database implementations.

```
class IVectorStore(Protocol):
    def search(self, embedding: list[float]) -> CacheLookupResult:
    ...

    def insert(self, entry: CacheEntry) -> None:
    ...
```

ChromaVectorStore

Concrete implementation of IVectorStore. It uses Chroma for storing vectors and performing nearest-neighbor search.

Responsibilities:

- Convert internal cache entries into Chroma records.
- Query Chroma for nearest embeddings.
- Interpret similarity/distance score.
- Return hit/miss result based on threshold.

IEvictionPolicy

Internal interface for eviction policies.

```
class IEvictionPolicy(Protocol):
    def choose_victim(self, entries: list[CacheEntry]) -> str:
    ...
```

Initial implementation:

- LRUEvictionPolicy

LLMServiceClient

A client side wrapper used by the orchestrator to communicate with the LLM Service process through gRPC.

It hides the generated gRPC stub details from the orchestrator business logic and exposes a simple interface for generating a LLM response.

It is used only when the vector store returns a cache miss.

LLMService

The gRPC server implementation inside the LLM Service process. It owns an ILLMProvider instance.

ILLMProvider

Internal interface for LLM providers.

```
class ILLMProvider(Protocol):
    def generate(self, prompt: str) -> str:
    ...
```

OllamaLLMProvider

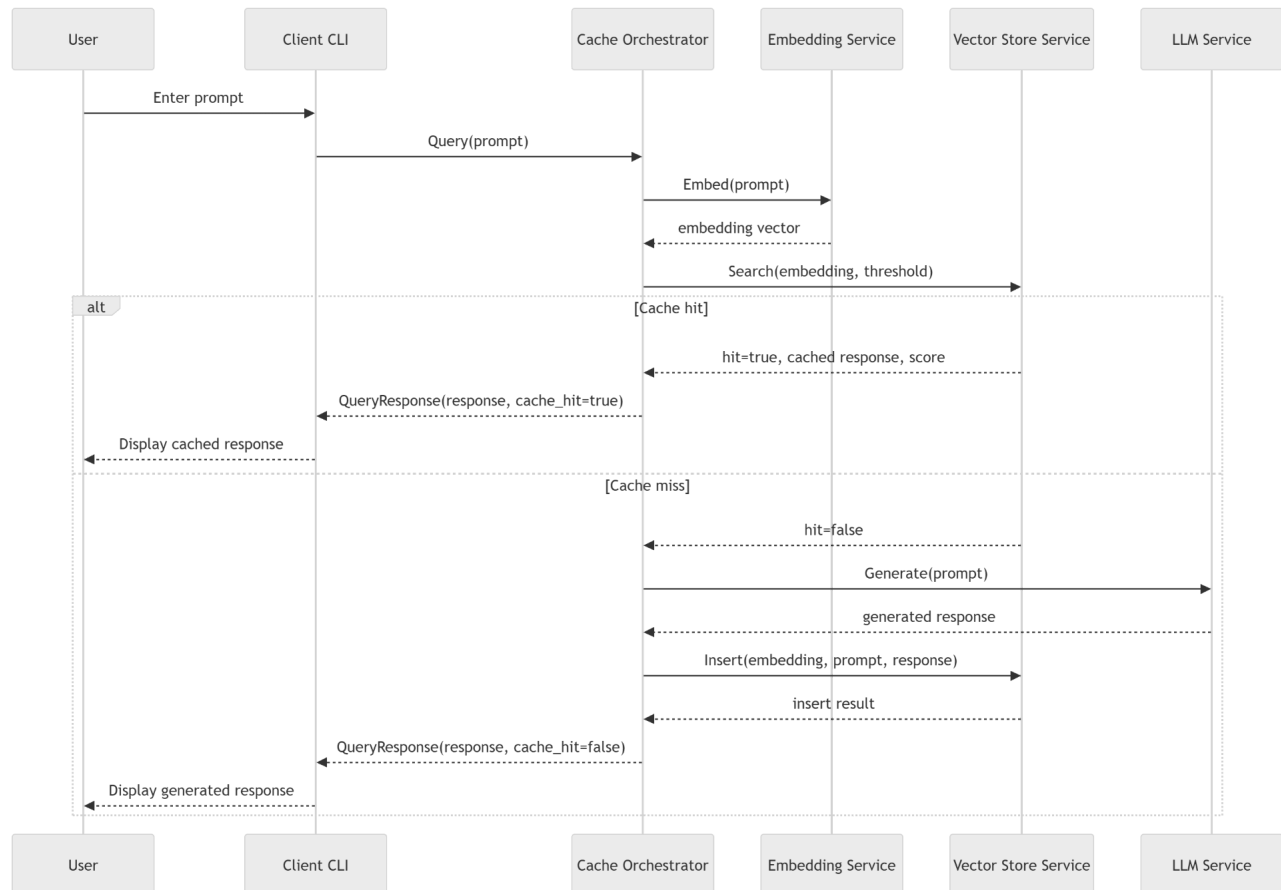
Concrete implementation of ILLMProvider. It calls Ollama using a configured model name.

Responsibilities:

- Hold Ollama endpoint and model name.
 - Send prompt to Ollama.
 - Return generated response text.
 - Convert provider-specific failures into service-level errors.
-

3.2 Flows

3.2.1 Main Query Flow



3.2.2 Cache Hit Example

1. User enters: What is semantic caching?
2. Client sends the prompt to the orchestrator.
3. Orchestrator requests an embedding.
4. Vector store finds an existing cached entry with similarity score 0.91.
5. Since $0.91 \geq \text{threshold } 0.85$, this is a cache hit.
6. Orchestrator returns the cached response with metadata: `cache_hit=true`.

3.2.3 Cache Miss Example

1. User enters a new prompt that is not close enough to any cached entry.
 2. Orchestrator requests an embedding.
 3. Vector store returns hit=false.
 4. Orchestrator sends the original prompt to the LLM Service.
 5. LLM Service returns a generated response.
 6. Orchestrator inserts (prompt, embedding, response) into the vector store.
 7. Orchestrator returns the response with metadata: cache_hit=false.
-

3.3 User Interface

The user interface is a command line application.

The user starts the application and receives a prompt input line. The user types a question and receives a textual answer. The CLI may also display optional metadata such as whether the result came from the cache or from the LLM.

Example:

```
$ python run_app.py --embedding-provider ollama --llm-provider ollama --vector-store chroma
```

LLM Cache CLI is ready.

Type your question or 'exit'.

> What is semantic caching?

[cache miss] Semantic caching is ...

> Explain semantic cache in simple words

[cache hit, score=0.89] Semantic caching is ...

3.4 Setup

3.4.1 Local Development Setup

Local development can run all services on the same machine as separate processes:

```
python -m services.embedding_service --provider ollama --port 50051
python -m services.vector_store_service --provider chroma --port 50052
python -m services.llm_service --provider ollama --port 50053
python -m services.orchestrator --config config/local.yaml --port 50050
python -m client.cli --orchestrator localhost:50050
```

3.4.2 Docker Compose Setup

For easier local execution, Docker Compose can start all services

The CLI may then connect to the orchestrator using the configured endpoint.

3.4.3 Unified Launcher

A later version may include a launcher script:

```
python run_app.py \  
  --embedding-provider ollama \  
  --llm-provider ollama \  
  --vector-store chroma
```

The launcher responsibilities:

- Load configuration.
- Decide which services are local and which are remote.
- Start local containers/processes when needed.
- Wait for health checks.
- Start the CLI after the required services are ready.

For remote services, the launcher should use configured endpoints instead of trying to start processes on another machine directly.

4. Use Cases

Use Case 1: Submit Prompt and Receive Cached Response

Primary actor: User

Goal: Receive an answer quickly from the semantic cache.

Preconditions: A similar prompt already exists in the vector store.

Main flow:

1. User submits a prompt through the CLI.
2. Orchestrator obtains embedding.
3. Orchestrator queries vector store.
4. Vector store finds a sufficiently similar cached entry.
5. Orchestrator returns cached response.
6. CLI displays response to user.

Postconditions: No LLM call is made. Cache metadata may be updated, for example last access time.

Use Case 2: Submit Prompt and Receive New LLM Response

Primary actor: User

Goal: Receive an answer for a prompt that is not in the cache.

Preconditions: No sufficiently similar cached entry exists.

Main flow:

1. User submits a prompt through the CLI.
2. Orchestrator obtains embedding.
3. Orchestrator queries vector store.
4. Vector store returns cache miss.
5. Orchestrator forwards prompt to LLM service.
6. LLM service returns generated response.
7. Orchestrator inserts new cache entry into vector store.
8. Orchestrator returns response to client.
9. CLI displays response to user.

Postconditions: A new embedding response pair is stored in the cache.

Use Case 3: Replace Embedding Provider

Primary actor: Developer

Goal: Change the embedding provider without changing the orchestrator.

Main flow:

1. Developer implements a new class that satisfies IEmbedder.
2. Developer updates Embedding Service configuration.
3. Embedding Service starts with the new provider.
4. Orchestrator continues calling the same gRPC Embed method.

Postconditions: The system uses the new embedding provider while preserving the same external architecture.

Use Case 4: Run Services on Different Machines

Primary actor: Developer / Operator

Goal: Deploy components on different hosts.

Main flow:

1. Operator starts each service on its target machine or container platform.
2. Operator configures orchestrator with service endpoints.
3. Orchestrator performs health checks.
4. Client connects to orchestrator.
5. Query flow works the same as in local deployment.

Postconditions: Components communicate through gRPC over the network.

Use Case 5: Cache Reaches Capacity

Primary actor: System
Goal: Keep cache size bounded.
Preconditions: Cache is full and a new entry should be inserted.
Main flow:

- 1. Orchestrator sends insert request to Vector Store Service.
- 2. Vector Store Service detects that capacity is full.
- 3. Vector Store Service applies the configured eviction policy.
- 4. One or more old entries are removed.
- 5. New entry is inserted.
- 6. Insert result is returned to orchestrator.

Postconditions: Cache remains within configured capacity.

5. References

Technology / Package	Planned Usage	What We Implement Around It
gRPC	Inter process communication	.proto service contracts, servers, generated clients
Protocol Buffers	Message serialization	Request/response schemas for all services
Ollama	Local LLM and embedding model runtime	OllamaEmbedder, OllamaLLMProvider
Gemma / Gemma family model	LLM response generation	Used through OllamaLLMprovider implementation
embeddingGemma or equivalent embedding model	Prompt embedding generation	Used through IOllamaEmbedder implementation
Chroma	Vector storage and similarity search	ChromaVectorStore implementation
Docker	Local multi process deployment	Compose files and service configuration
Python	Main implementation language	Service implementations, CLI, launcher